

Observe, Expand, Recover: An Observability-Guided Agent for Safe Query Expansion

Author One*, Author Two†

*Affiliation One, email@example.com

†Affiliation Two (AI Observability), email2@example.com

Abstract—Large language model (LLM) query expansion is typically applied either unconditionally or not at all, despite evidence that unconditional expansion is net-harmful on a non-trivial fraction of queries. We formulate query expansion as a bounded, observable agentic decision, evaluated under a strictly separated validation/test protocol: every retrieval-config, verifier-model, and threshold decision is fixed on TripClick TAIL validation before a single frozen touch of test. An under-tuned BM25 baseline (nDCG@10 0.228), corrected via validation-only stemming and title weighting, reaches 0.315, above the published reference, changing every downstream conclusion. Pre-action telemetry predicts whether retrieval is already poor with ROC-AUC 0.649, but the same telemetry is markedly worse at the harder, actionable question — will a *specific* expansion help — at ROC-AUC only 0.552 ($r = 0.114$, $p < 0.001$). Detection is not actionability, and that gap is this paper’s central finding. Consistent with it, neither static expansion nor a closed-loop agent significantly outperforms corrected BM25 (static: +0.0007, agent: +0.0009 nDCG@10), nor does the agent beat static; decomposing the pipeline shows reranking and expansion are each significantly *harmful* alone but interact positively together, not one canceling a fixed cost from the other. This is not for lack of headroom: an oracle gains +0.0246 nDCG@10 over BM25 ($p < 0.001$), of which the verifier captures only 0.5–12% across intervention budgets. Fault-injection recovery is fault-specific, causally explained by which telemetry features each fault perturbs — one fault even *increases* the verifier’s healthy-state score rather than merely evading detection. A diagnostic rule built from three faults correctly anticipated a fourth, held-out fault’s detection/false-alarm regime before it was run, though whether detection converts to a significant recovery gain remains fault-specific (confirmed for only one of four). Generalization is worse outside the calibration split: action-utility AUC falls to 0.148 (HEAD) and 0.214 (TORSO), below chance. We report a real implementation defect found mid-study and its correction. Runtime observability reliably detects retrieval failure and, for large structured faults, indicates in advance how the verifier will behave toward a new failure class; but the stronger property general autonomous recovery needs — predicting whether a specific intervention on an ordinary query will help — is not delivered by the same telemetry, under a protocol designed so these findings are trustworthy.

Index Terms—query expansion, information retrieval, agentic systems, AI observability, retrieval-augmented generation, large language models, site reliability engineering

I. INTRODUCTION

Large language models can generate useful pseudo-documents, alternative phrasings, and disambiguating context for a search query [1], [2], and corpus-grounded generation — conditioning the rewrite on retrieved evidence rather than the query text alone — improves

on naive query-only rewriting. Existing work establishes that LLM query expansion *can* help, not when an operational system should trust an expansion enough to use it. In practice, expansion is applied unconditionally or not at all; neither is safe, since unconditional rewriting is reliably net-harmful on a measurable fraction of queries, particularly underspecified or already-well-served ones.

We treat this as a control problem rather than a generation problem. Borrowing the observe-diagnose-act-recover discipline of site reliability engineering [3] and the trace/metric vocabulary of OpenTelemetry [4], we build an agent whose action space is a small set of interpretable, corpus-grounded expansion operators, whose control loop alternates actions with structured observations as in ReAct [5], and whose accept, roll-back, replan, and abstain decisions are driven by runtime telemetry rather than the language model’s own self-assessment. The central question is not whether an LLM can produce a useful expansion — that is established — but whether *runtime observability* can make expansion selective, recoverable, and cost-aware. Answering it surfaces a distinction the selective query expansion and query performance prediction literature has not, to our knowledge, isolated: predicting that retrieval is failing and predicting that a specific intervention will fix it are different tasks, and telemetry good at the first can be useless at the second. This paper reports what we found, including where the answer was no.

A. Research Questions

RQ1 (Detection vs. Actionability). Can qrel-free retrieval and agent-runtime telemetry (a) identify that unexpanded retrieval is already unhealthy, and (b) predict whether a *specific* expansion action is likely to help or harm — and, if both are possible, is the second any easier than the first once the first is solved?

RQ2 (Control). Does an agent that conditions expansion actions on that telemetry outperform an otherwise identical *static*, failure-conditioned pipeline — same retriever, grounding, generator, and fusion, differing only in whether a closed loop can observe an action’s consequences and recover from a bad one?

RQ3 (Recovery). After a harmful rewrite or a simulated system-level fault, can observability-driven verification detect the degradation and select a recovery action to restore retrieval quality, under a bounded budget — even for a verifier whose general-purpose detection power (RQ1) is only weakly informative?

B. Contributions

We contribute (1) an agent architecture treating LLM query expansion as a bounded, typed-tool action inside a closed observability loop, under a frozen train/validation/test protocol (Section IV-B) where every retrieval-config, verifier, and threshold decision is made on validation before one single-touch test evaluation; (2) a corrected BM25 baseline, selected entirely on validation, moving our TripClick TAIL retriever above the published reference and changing every downstream conclusion; (3) a direct comparison, from one telemetry vocabulary, of failure-*detection* against intervention-*utility* prediction,

showing these differ and only the first is reliable; (4) a frozen-test evaluation of the closed-loop agent against a matched static comparator, with an oracle and selectivity-frontier analysis quantifying captured headroom; (5) a fault-injection study with a causal feature-perturbation analysis and a diagnostic rule, confirmed *prospectively* on a held-out fault never used to derive it — telemetry can indicate whether a new failure class will be over-flagged, under-flagged, or genuinely discriminable before it is ever observed, not just explain failures already seen; (6) a cross-split check showing that ranking degrades below chance outside its calibration split; and (7) an honest account of a real implementation defect, corrected mid-study.

II. RELATED WORK

LLM query expansion. Query2doc [1] generates pseudo-documents from an LLM and appends them to the query; MuGI [2] generates and fuses multiple expansions. Both treat expansion as unconditional, one-shot generation, conditioning on neither retrieval-state telemetry nor a harmful rewrite’s recovery.

Agentic control and observability. ReAct [5] interleaves reasoning with actions and observations, the template for our observe-act-observe structure, but with a much smaller action space: typed retrieval operators, keeping the policy interpretable under a non-LLM controller. OpenTelemetry’s trace/metric/log separation and GenAI semantic conventions [4] supply a naming discipline for our telemetry; Google’s SRE framing of an SLO as a target range, not a metric to optimize [3], motivates our Agent Search SLO: quality subject to harm, cost, and latency frozen *before* evaluation.

Query performance prediction and selective expansion. Classic query performance prediction [6] and selective query expansion [7] already ask whether a query is hard, or likely to benefit from expansion, from retrieval signal alone. We split that into two predictive targets sharing one telemetry vocabulary — is retrieval failing, versus will *this* action fix it — and show qrel-free telemetry answers the first far better (Section V).

Benchmarks. TripClick [8] has genuine head/torso/tail query-frequency structure, unlike benchmarks that treat “hard” as a synonym for low-BM25-score, which is why it is our sole evaluation corpus.

III. SYSTEM DESIGN

A. Agent State and Control Loop

The agent’s state at step t is $s_t = (q_0, q_t, R_t, O_t, H_t, B_t)$, where q_0 is the immutable original query, q_t the active search expression, R_t the current ranking, O_t the observability state, H_t the episode’s action/observation history, and B_t the remaining action budget. The control loop is OBSERVE $\rightarrow$ DIAGNOSE $\rightarrow$ ACT $\rightarrow$ RETRIEVE $\rightarrow$ OBSERVE $\rightarrow$ VERIFY $\rightarrow$ {ACCEPT, ROLLBACK, REPLAN, STOP}. The critical property this loop is built to test is that the second decision (accept/rollback/replan/stop) depends on evidence the agent’s own first action created — as opposed to a static pipeline that receives the same query, initial ranking, evidence, generator, and verifier, but executes exactly one predetermined route. Isolating this dependency, not simply comparing against unmodified BM25, is the paper’s central experimental contrast (Section VI).

The action space is deliberately small and interpretable: $A = \{ \text{NOEXPAND, VOCABULARY, CONTEXT, AMBIGUITY, ENTITYNORMALIZE, ROLLBACK, REPLAN, STOP} \}$. VOCABULARY maps informal user language onto corpus-compatible terminology; CONTEXT adds a corpus-supported missing facet to an underspecified query; AMBIGUITY disambiguates toward the interpretation the evidence best supports; ENTITYNORMALIZE resolves aliases, abbreviations, and canonical-name mismatches. The LLM is invoked through exactly one typed tool, `generate_expansion(action, query, evidence)`; it is never the sole arbiter of its own output’s use — that judgment is a separate, non-generative controller’s (Section III-D).

B. Observability Model

Every query episode produces one trace with spans `retrieve_original` $\rightarrow$ `build_evidence` $\rightarrow$ `plan_action` $\rightarrow$ `generate_expansion` $\rightarrow$ `retrieve_expanded` $\rightarrow$ `rerank` $\rightarrow$ `verify` $\rightarrow$ `recover_or_accept` $\rightarrow$ `fuse`. O_t spans three telemetry families. *Retrieval*: top score, score margin, normalized score entropy, query-term coverage, mean rare-term IDF, entity overlap, top- k ranking overlap against the previous state, reranker/BM25 disagreement, lexical result coherence, and ranking stability under a small controlled query perturbation. *Agent*: selected operator, trajectory length, previous verifier score, rewrite-to-original semantic drift, introduced entities, repeated actions, remaining budget. *System*: per-tool latency, token counts, tool errors/timeouts. Every feature is computable without relevance judgments, which is what makes the verifier usable at inference time.

C. Failure Taxonomy

We label episodes into ten failure categories (vocabulary mismatch, underspecification, ambiguity, entity mismatch, intent drift, confirmation bias, retriever disagreement, tool degradation, budget exhaustion, healthy retrieval), each with a telemetry signature and preferred recovery action, motivating Section VII’s fault-injection protocol.

D. Verifier and Controller

The verifier, `verify(previous_state, new_state)`, produces a scalar judgment of whether an action’s consequences look like an improvement, using telemetry alone. Rather than hand-picking its weights, we calibrate them against click-derived qrels (Section IV): execute one expansion action, compute the true nDCG@10 change against those labels, and fit the verifier’s weights to predict that change from Section III-B’s telemetry features. Click labels are available only during this offline calibration; the deployed verifier never sees one, and is treated as an artifact — trained by a script, loaded at inference time. The recovery controller is a small deterministic finite-state machine: below a calibrated harmful threshold the agent rolls back to the pre-action ranking; above a calibrated accept threshold it fuses the new ranking via reciprocal-rank fusion; in between it replans with an untried operator, capped at two model calls per query.

IV. EXPERIMENTAL SETUP

A. Datasets

TripClick [8] is the primary benchmark: 1,523,878 documents, split evenly across head/torso/tail frequency buckets, each further split into train/validation/test (1,175 queries per split per bucket). All results concern the TAIL split unless stated otherwise (Section VIII). Labels are click-derived, not human-assessed (binary raw clicks, matching the label type this split’s published reference point uses [8]); we use “qrel” and “click label” interchangeably, neither implying editorial judgment.

B. Experimental Protocol

A first pass at this study calibrated the verifier and thresholds on the same episodes later used to report RQ1–RQ3 — an in-sample characterization unsuited to an out-of-sample generalization claim. We froze a stricter protocol instead, in a committed config rather than left implicit: TAIL *validation* may be used for any retrieval-config, verifier, or threshold decision; TAIL *test* may be touched at most once per research question, purely for evaluation, after every upstream decision is frozen, with no decision afterward conditioned on a test-set outcome. This discipline directly produced the paper’s first finding, below.

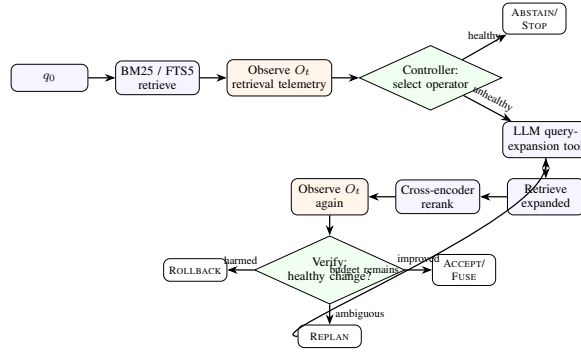


Fig. 1. The observability-guided control loop. Telemetry O_t (orange) drives both operator selection and the accept/rollback/replan/stop decision. The static comparator (Section VI) runs the same chain once, with no path back from VERIFY to the LLM tool.

C. A Corrected Retrieval Baseline

BM25 over an unstemmed FTS5 index scored nDCG@10 0.228 on TAIL test — 15% below the 0.267 reference for this exact split and label type [9]. Selecting entirely on validation, we found two config choices that close this gap: Porter stemming (val nDCG@10 0.235 $\rightarrow$ 0.270) and weighting the title field ten times body weight in a title/body-split FTS5 index (val nDCG@10 climbing to a clear plateau at that ratio, 0.327, MRR peaking there and declining beyond it). A single, one-time sanity check of the frozen config against withheld test — not a selection criterion — gives nDCG@10 0.315, *above* the published reference. We report this because an under-tuned baseline mechanically inflates the apparent value of anything compared against it: every downstream conclusion in this paper changed once this fix was in place, a precondition for the rest of the study rather than an implementation footnote.

D. Implementation

Sparse retrieval uses BM25 over a SQLite FTS5 index [10] rather than a JVM Lucene index, forced by a JVM crash under Pyserini [11] on our hardware; two further silent-corruption risks surfaced at this corpus’s scale, the same category as Section IV-C’s baseline gap and Section VII-A’s telemetry defect: FTS5’s default MATCH operator is *conjunctive* (collapsing recall without an error), and FTS5 indexes only the MATCH column (an unindexed companion lookup degrades to a full table scan, $18\times$ slower at 1.5M rows). Cross-encoder reranking (cross-encoder/ms-marco-MiniLM-L6-v2 [12]) applies to the top 100 BM25 candidates, reranking the top 50. Generation uses a locally hosted, 4-bit quantized 2–3B-parameter model, capped at two calls per query episode with a 32–64 token structured output, so the language model is never the entire decision surface, only one typed tool within it.

E. Verifier Calibration Methodology

Following Section IV-B, we collect calibration episodes (one expansion action per query, true nDCG@10 change against click labels) on TAIL *validation* ($n = 1147\text{--}1168$), and compare three feature families of increasing size (six retrieval-telemetry deltas; plus four reranker/length features; plus three dense-embedding features) against three model classes each (fixed-weight, 5-fold CV linear, 5-fold CV GBDT), under Pearson correlation (Table I), to characterize the detection ceiling rather than accept the first plausible fit — never itself reported as evidence the signal generalizes (Section V does that separately, from one frozen touch of test). Recovery thresholds are calibrated the same way, from candidate cutoffs on validation.

F. Metrics

Retrieval quality is nDCG@10, Recall@100, and MRR, plus the fraction of queries helped/unchanged/harmed relative to a baseline.

TABLE I

VALIDATION CHARACTERIZATION OF CANDIDATE VERIFIER FAMILIES ON TAIL *validation* ($n = 1147\text{--}1168$): PEARSON r VS. TRUE NDCG@10 CHANGE. NOT THE GENERALIZATION RESULT (TABLE II).

Model, feature family	Val Pearson r
v0 hand-picked weights (6 features)	+0.020 (n.s., $p=0.50$)
Linear, 10 features (lexical+reranker)	−0.014
GBDT, 10 features	+0.001
Linear, 13 features (+dense embedding)	−0.015

TABLE II

RQ1’S FROZEN RESULTS: BOTH MODELS EVALUATED *once* ON WITHHELD TAIL TEST ($n = 1157$), NEVER TOUCHED DURING SELECTION.

Metric	Test value
Action utility: Pearson r vs. true Δ nDCG@10	0.114 ($p < 0.001$)
Action utility: ROC-AUC (detecting harm)	0.552
Action utility: average precision (prev. 0.330)	0.364
Failure detection: ROC-AUC (val CV 0.618)	0.649

For the recovery study (Section VII) we report Detection Recall $P(\text{intervened} \mid \text{harmful})$, False Alarm Rate $P(\text{intervened} \mid \text{healthy})$, and recovery gain in mean nDCG@10 with vs. without recovery on identically corrupted episodes. We additionally report a retrospective *oracle ceiling*: per query, $\max(\text{baseline}, \text{treatment})$ under true click labels, diagnosing whether headroom exists independent of whether our qrel-free verifier can find it (Section VI). Significance is via paired bootstrap (10,000 resamples) throughout.

V. RQ1: DETECTION

Nothing clears noise on validation: every candidate — a hand-picked baseline, linear and gradient-boosted fits on the deployed 10-feature set, and a fit augmenting it with dense-embedding features — lands at or below $|r| = 0.02$. We select the 10-feature linear model, the only candidate fit on the corrected pipeline’s feature distribution rather than a stale or untested one (configs/experimental_protocol.yaml) — though that criterion alone does not distinguish it from GBDT-10, fit on the identical distribution and statistically indistinguishable from it ($r=+0.001$ vs. -0.014); linear is also the only class compatible with Section VII’s closed-form attribution, though that was not the stated selection criterion at freeze time. We deploy it *despite* its weak validation estimate, per Section IV-B: no decision here is conditioned on test performance.

The frozen action-utility model generalizes better than its noisy validation estimate suggested: $r = 0.114$ on test is small but real ($p < 0.001$, $R^2 \approx 1.3\%$), modestly above no-skill baselines (Figure 2). We read the val/test gap honestly rather than as vindication: cross-validation on $\sim 1,150$ episodes is a noisy estimator for an effect this

TABLE III

RQ2, FROZEN RESULT, FULL TRIPCLICK TAIL TEST ($n = 1175$), CORRECTED BASELINE. STATIC QE AND AGENT SHARE RETRIEVER, GROUNDING, GENERATOR, VERIFIER, FUSION; AGENT ADDS A CLOSED OBSERVE-VERIFY-RECOVER LOOP.

System	nDCG@10	Recall@100	MRR
BM25 only	0.3152	0.7750	0.3014
BM25 + rerank, no QE	0.2850 [†]	—	—
BM25 + QE, no rerank	0.2992 [‡]	0.7716	0.2856
Static QE (QE + rerank)	0.3159	0.7759	0.2993
Agent	0.3161	0.7770	0.3015

[†]: significantly below BM25 only, -0.0302 [$-0.0447, -0.0154$], $p < 0.001$. [‡]: significantly below BM25 only, -0.0160 [$-0.0237, -0.0086$], $p < 0.001$ (scripts/13).

small, and one larger test set landing positive does not retroactively make the selection sound.

A. Detection Is Not Actionability

That action-utility model answers a hard question: given telemetry before *and* after a specific action, did it help? A logically prior, cheaper question uses only pre-action telemetry: is unexpanded retrieval already unhealthy? We build a second model from the same eight retrieval-telemetry features, labeling a query “poor” when BM25 surfaces zero relevant documents in the top 10 (test prevalence 0.450), selected by 5-fold CV on validation and touched once on frozen test, identically to Section IV-E’s protocol — Table II’s last row.

The gap is large and consistent across val and test: telemetry genuinely informative about *whether* retrieval is failing ($AUC = 0.649$) is markedly weaker at predicting whether a *specific candidate action* fixes it ($AUC = 0.552$) — both frozen single-touch test numbers from the same features and protocol, not the same result at different noise levels. **RQ1’s answer:** qrel-free telemetry reliably detects that something is wrong, but is only weakly informative about whether a proposed fix will work — the gap RQ2’s control result and RQ3’s fault-specific pattern trace back to.

VI. RQ2: CONTROL

Once BM25 is corrected (Section IV-C), the picture changes entirely: neither Static QE nor Agent significantly outperforms BM25 alone (static: $+0.0007$ nDCG@10, 95% CI [$-0.0066, 0.0079$]; agent: $+0.0009$, 95% CI [$-0.0069, 0.0087$]), and Agent does not significantly outperform Static QE ($+0.0002$, 95% CI [$-0.0027, 0.0032$]; changes 3.9% of outcomes, 2.1% helped, 1.7% harmed). The agent still costs 24% more model calls (1.22 vs. 0.98 per query) and 36% higher mean latency (15.1 s vs. 11.1 s, excluding sleep-inflated episodes) for no offsetting gain. Decomposing the two components (Table III) shows an interaction, not one offsetting a fixed cost from the other: reranking alone is significantly *harmful* (-0.0302 , $p < 0.001$) and expansion alone is *also* significantly harmful (-0.0160 , 95% CI [$-0.0237, -0.0086$]) — neither component helps in isolation. Yet adding the reranker back once expansion is already applied significantly *helps* ($+0.0167$, 95% CI [$0.0076, 0.0257$]), the opposite sign from adding it to raw BM25: the cross-encoder, trained on general web relevance, misorders raw BM25’s candidate pool but behaves better once expansion has already reshaped it. Static QE’s near-parity with BM25 is this interaction landing close to neutral, not one component compensating for the other. **On natural queries, neither expansion flavor moves the needle once the retriever is competitive** — materially different from what a weak baseline would have supported.

A. Is There Any Headroom at All?

A null result here is ambiguous: either no control opportunity exists, or one exists but this verifier cannot find it. We resolve this

TABLE IV

UNCONSTRAINED ORACLE CEILING, SAME EPISODES AS TABLE III. ORACLE PICKS $\max(\text{BM25}, \text{TREATMENT})$ PER QUERY, TRUE CLICK LABELS, NO BUDGET CAP.

Comparison	nDCG@10 gain	95% CI
Oracle-static vs. BM25	$+0.0246$	[0.0197, 0.0296]*
Oracle-static vs. Static QE	$+0.0239$	[0.0191, 0.0292]*
Oracle-agent vs. Agent	$+0.0276$	[0.0226, 0.0332]*

*: significant, $p < 0.001$.

TABLE V

SELECTIVITY FRONTIER, TAIL TEST ($n = 1175$, SCRIPTS/14): MEAN NDCG@10 GAIN OVER ALWAYS-BM25 WHEN THE TOP $X\%$ OF QUERIES BY POLICY RANK GET TREATMENT, REST AT BM25. BUDGET-ORACLE IS THE SAME ORACLE AS TABLE IV, CAPPED TO THE TOP $X\%$; IT NECESSARILY PLATEAUS AT TABLE IV’S UNCONSTRAINED $+0.0246$ ONCE THE BUDGET COVERS EVERY QUERY THE ORACLE WOULD TREAT. % COLUMN: VERIFIER GAIN AS A FRACTION OF BUDGET-ORACLE GAIN AT THAT BUDGET.

Budget	Verifier gain	Budget-oracle	%
5%	0.0001	0.0184	0.5%
10%	0.0006	0.0242	2.5%
20%	0.0025	0.0246	10.2%
40%	0.0029	0.0246	11.8%
100%	0.0007	0.0007	100%*

* At 100% budget every policy treats every query, so ranking cannot matter; verifier and oracle necessarily coincide with Table III’s unconditional Static QE result.

with a retrospective oracle: per query, take whichever of BM25 or the treatment ranking true click labels say is better — not deployable, but useful to bound the second explanation.

The answer is unambiguous: substantial headroom exists. A perfect selective controller would gain $+0.0246$ nDCG@10 over BM25 — roughly $35\times$ what Static QE actually captures ($+0.0007$) — significant at $p < 0.001$. Static expansion helps 11.6% of queries and harms 10.6% relative to BM25, a real, roughly-balanced mix a perfect selector could exploit. The failure is squarely in *detection*: RQ1’s verifier, $r = 0.114$ on frozen test, is far too weak to discriminate the two groups — converting RQ2’s null result from “maybe there’s nothing here” into “there is something here, and our qrel-free detector cannot see it.”

At every budget below universal treatment, the verifier captures only a small fraction of available headroom — 0.5% at 5%, rising to 11.8% at 40%, never close to the budget-oracle: far short of what perfect knowledge extracts.

Table VI shows a controller that intervenes often (38.9% ROLL-BACK or REPLAN) without those interventions paying off: the frozen thresholds are wide, since nothing on validation cleanly separated harmful from healthy episodes, so the controller second-guesses frequently but not accurately. **RQ2’s answer is no**, with the oracle clarifying *why*: an observability-guided closed loop built on this verifier does not outperform a matched static pipeline, not because no opportunity exists, but because this verifier cannot exploit it.

VII. RQ3: WHICH FAILURES ARE RECOVERABLE?

RQ1 and RQ2 leave open whether observability has *any* practical value, or whether a weak general detector is useless. We test a narrower claim: that the same weak verifier can serve as a safety net against *large, obvious* failures even though it cannot reliably judge *subtle, natural* ones. We inject three fault types at their natural point of corruption — an unsupported entity appended to the expansion, a grounding passage swapped for a non-supporting one, and the reranker disabled — and compare, on identical corrupted episodes, an always-accept ablation against the real controller.

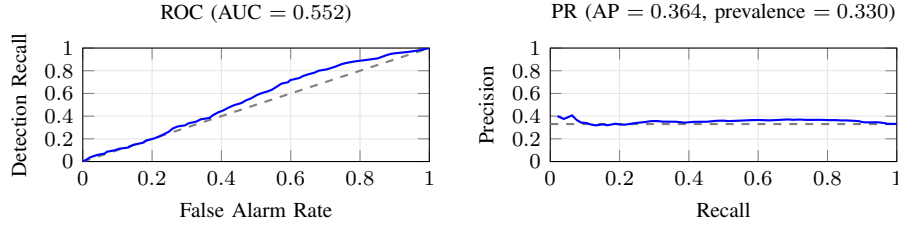


Fig. 2. RQ1’s frozen result: ROC and precision-recall curves for the selected verifier’s score against true harm (nDCG@10 delta < 0) on withheld TAIL test ($n = 1157$, single touch). The curve sits consistently, but only modestly, above the no-skill baseline (dashed) in both panels.

TABLE VI
AGENT CONTROLLER-DECISION FREQUENCY, FULL TRIPCLICK TAIL TEST SET ($n = 1175$).

Decision	Count	%
ACCEPT	699	59.5
ROLLBACK	331	28.2
REPLAN	127	10.8
ABSTAIN (no expansion)	18	1.5

TABLE VII
RQ3, FROZEN RESULT: FAULT INJECTION, CORRECTED PIPELINE, TAIL TEST ($n = 1157$; 18/1175 SKIPPED, NO EXPANSION). GAIN IS MEAN NDCG@10 WITH MINUS WITHOUT RECOVERY ON IDENTICALLY CORRUPTED EPISODES; CI FROM 10,000-RESAMPLE BOOTSTRAP. DOMINANT CONTRIBUTOR: FEATURE j WITH LARGEST $|w_j \Delta x_j|$ (BELOW).

Fault type	Rec.	FA	Gain (95% CI)	Dominant contrib.
Entity injection	0.974	0.939	+0.0081 [-0.003, +0.019]	coverage_gain -0.054
Grounding corruption	0.528	0.469	+0.0099 [+0.002, +0.018]*	topk_overlap +0.002
Reranker disabled	0.000	0.002	+0.0000	disagreement +0.126

CIs straddle zero except *: significant, positive. FA: false alarm.

TABLE VIII
NET CONTRIBUTION PREDICTS RECALL/FALSE-ALARM REGIME, NOT GAIN: NEAR-ZERO IS WHERE REAL DISCRIMINATION HAPPENS.

Fault	Net contrib.	Recall	FA	Gain
Entity injection	-0.0529	0.974	0.939	NS
Query-term removal†	-0.0236	0.800	0.694	NS
Grounding corruption	+0.0002	0.528	0.469	*
Reranker disabled	+0.1296	0.000	0.002	NS

†: held out, predicted before running (below). *: significant, $p < 0.05$. NS: not significant.

reranker-derived features, it makes the corrupted state score higher, so the controller is actively misled, not merely blind. Section VII-C generalizes this causal signature into a rule and tests it prospectively.

RQ3’s answer is that recoverability is fault-specific, and the attribution shows why: content-level corruption produces a real net gain only when its effect on the verifier’s score is small enough to leave room for genuine per-episode discrimination (grounding), is a wash when the effect is large and uniform enough to flag almost everything (entity injection), and a system-level fault is disguised as an improvement when it moves a strongly-weighted feature the *wrong* way (reranker) — a verifier is not uniformly weak or strong across failure types, it can be actively adversarial to a fault it was never calibrated against. This pattern is not just post-hoc description, either: Section VII-C tests it prospectively.

C. A Diagnostic Rule, Confirmed on a Held-Out Fault

The three faults’ net contributions ($\sum_j w_j \Delta x_j$) and their recall/false-alarm pairs move together: strongly negative (entity injection, -0.0529) pushes the score down almost regardless of true harm, giving high recall but comparably high false alarms; strongly positive (reranker disabled, +0.1296) does the opposite (recall/FA near zero); only near-zero (grounding, +0.0002) leaves room for per-episode variation to separate harmful from healthy cases — the one fault with a significant net gain. This is usable *before* a new fault is ever run: reason about which features a candidate corruption should perturb, weight by the frozen verifier’s coefficients, and read off whether it will be over-flagged, under-flagged, or genuinely discriminable.

We tested this prospectively on a held-out fourth fault never used to derive the rule: `remove_high_idf_term`, dropping the query’s single most discriminative term before retrieval — mechanistically distinct from all three. Predicted in writing before running: negative net contribution, an over-flagged profile like entity injection. Result: net contribution -0.0236, recall 0.800, false alarm 0.694 — the predicted regime, confirmed, though the dominant feature was not (`topk_overlap`, not the predicted `coverage_gain`; `scripts/12`). **The verifier’s fixed weights are diagnostic for how a genuinely new failure class will behave, not merely descriptive of the three faults they were fit against**: whether the verifier will over-flag it, under-flag it, or discriminate it is predictable in advance, even though whether that discrimination converts into a significant recovery gain (as it did only for grounding corruption) still requires running it, and the per-feature mechanism does not always transfer.

A. A Defect That Changed the Answer

An early pass found the reranker-disabled fault essentially undetectable (recall 0.185) while the other two looked strong. Tracing this revealed a real defect: the reranked ranking was computed, at real latency cost, but never actually applied to the ranking the agent accepted — disabling the reranker was mechanically a no-op regardless of verifier quality. Fixing it exposed a second, entangled defect: the verifier’s telemetry was still computed from the pre-rerank ranking. We corrected both and re-ran calibration to confirm the deployed verifier did not need retraining (corrected Pearson $r = 0.146$ vs. the original 0.134). We report this because a component computed purely for observability is architecturally easy to leave disconnected from the outcome it explains, and the resulting error looked coherent — unrealistically strong — until an implausible near-zero effect prompted investigation.

B. Frozen Results

The three fault types split into three distinct outcomes: no single answer to “is recovery useful,” only a per-failure-type one. **Entity injection** is detected almost every time (recall 0.974) but flagged almost indiscriminately (false alarm 0.939), netting out non-significant. **Grounding corruption** is the one clean recovery success (+0.0099, $p < 0.05$). **Reranker disabled** is never detected at all (recall 0.000): the controller essentially always accepts.

Why, causally. Fitted weights alone show which features the verifier ignores, not what a fault *does* to its score: $\Delta x_j = \mathbb{E}[x_j^{\text{fault}} - x_j^{\text{clean}}]$. $\text{Contribution}_j = w_j \cdot \Delta x_j$ ($n=79/\text{fault}$; Table VII, last column). The clearest case is reranker disabled: its dominant contribution is large and *positive*, from `disagreement` moving in the direction the verifier reads as improvement — the fault does not merely evade weak

TABLE IX

CROSS-SPLIT ROBUSTNESS, FULL TEST SPLITS ($n = 1175$ EACH), FROZEN VERIFIER UNCHANGED.

Split	BM25	Static QE	Oracle gain	AUC
HEAD	0.2282	0.2293 (NS)	+0.0153*	0.148
TORSO	0.2485	0.2506 (NS)	+0.0187*	0.214
TAIL	0.3152	0.3159 (NS)	+0.0246*	0.552

NS: CI crosses zero. *: significant, $p < 0.001$. AUC: frozen verifier’s action-utility AUC on that split.

VIII. DOES THE VERIFIER GENERALIZE BEYOND TAIL?

Every calibration decision so far used TAIL validation only (Section IV-B); we did not test whether the frozen verifier’s action-utility ranking holds on HEAD and TORSO, which it never saw during selection. Using the identical frozen config, no recalibration, we run BM25 and Static QE on each bucket’s test split.

Static QE stays statistically indistinguishable from BM25 on every bucket, matching TAIL; oracle headroom exists on every bucket too. What does not transfer is the verifier: action-utility AUC falls from 0.552 on TAIL to 0.214 on TORSO and 0.148 on HEAD — both *below* chance, anti-correlated with true utility, not merely uninformative. A verifier calibrated on one frequency regime does not just fail to add value in another; its ranking inverts.

IX. DISCUSSION

Read together, RQ1–RQ3, the oracle analysis, the diagnostic rule, and the cross-split check sketch one finding. Section V-A is load-bearing: the same telemetry detects that retrieval is failing reasonably well (AUC 0.649) but is weakly informative about whether a specific action on an ordinary query will fix it (AUC 0.552). RQ2 shows the consequence — a closed loop built on the weaker signal adds nothing over a matched static pipeline — and the oracle and selectivity-frontier analyses rule out “no opportunity exists”: real headroom is available (+0.0246 nDCG@10) and the verifier reaches at most 12% of it. RQ3’s causal attribution shows this is a per-fault property, sharp enough in one case that the verifier actively reads the fault as an improvement. The diagnostic rule (Section VII-C) is the paper’s sharpest positive result precisely because it operates in this narrower regime: for *large, structured* faults, the same weights predict, before a new fault is run, which recall/false-alarm regime it falls into. Section VIII bounds this differently: holding the fault fixed and changing query frequency, the same ranking goes from weakly informative on TAIL to below-chance on HEAD/TORSO. Two generalization questions, opposite answers: new failure types within the calibration split, yes; the same behavior outside it, no.

The BM25 baseline and reranker-wiring defects (Sections IV-C, VII-A) were both found only by refusing a convenient result: an error that looks coherent rather than broken. A frozen protocol with a mandatory sanity check defends against the first kind; the second we only found by distrusting a result that seemed too good.

X. LIMITATIONS

The oracle ceiling (Section VI-A) is retrospective, using test-set click labels directly — a valid upper bound, never a deployable method. The generator is a locally hosted, quantized 2–3B parameter model under a two-call budget; a larger generator might change RQ1’s ceiling or RQ2’s null result. Cross-split robustness (Section VIII) covers TripClick’s three buckets only; other benchmarks remain untested, so the AUC collapse could be frequency-specific or reflect broader corpus-shift sensitivity. Click-derived qrels are an imperfect relevance proxy inherited from the benchmark. The diagnostic rule (Section VII-C) predicted the held-out fault’s recall/false-alarm regime correctly but not its dominant feature, and rests on one held-out fault, not a systematic sweep. The failure-detection model uses one binary definition of “poor retrieval”; a graded definition could shift its measured AUC.

XI. CONCLUSION

We evaluated a closed-loop, observability-guided query-expansion agent under a corrected protocol separating model selection from final evaluation — surfacing the paper’s first finding: an honestly-selected BM25 fix inverted every downstream conclusion. Runtime observability reveals retrieval failures reliably (detection AUC 0.649), but autonomous recovery needs a stronger property the same telemetry does not deliver: predicting whether a specific intervention will help (action-utility AUC 0.552, below chance outside its calibration split). That gap is why the closed-loop agent does not outperform static expansion and neither outperforms BM25, despite real headroom going uncaptured. The recovery study shows this is a per-fault, mechanistically explicable property — in one case the verifier is actively misled, not merely blind — and, tested prospectively on a held-out fault, this mechanism supports genuine diagnosis in advance: predicting, before a new large structured failure is run, whether the verifier will over-flag it, under-flag it, or discriminate it well, though whether that translates into a significant recovery gain still requires running it. That capability does not extend across query-frequency regimes, where the same ranking inverts outside its calibration split. We report both implementation defects found by refusing a convenient result, in the belief that the discipline to catch them is worth more than a flattering number.

ACKNOWLEDGMENT

The authors thank the TripClick maintainers for access to the dataset under its non-commercial research license.

REFERENCES

- [1] L. Wang, N. Yang, and F. Wei, “Query2doc: Query expansion with large language models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.585/>
- [2] L. Zhang, Y. Wu, Q. Yang, J. Lin *et al.*, “Exploring the best practices of query expansion with large language models,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. [Online]. Available: <https://aclanthology.org/2024.findings-emnlp.103/>
- [3] Google, “The site reliability workbook: Defining SLO,” 2026. [Online]. Available: <https://sre.google/sre-book/service-level-objectives/>
- [4] OpenTelemetry Authors, “OpenTelemetry signals and GenAI semantic conventions 1.44.0,” 2026. [Online]. Available: <https://opentelemetry.io/docs/concepts/signals/>
- [5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [6] S. Cronen-Townsend, Y. Zhou, and W. B. Croft, “Predicting query performance,” in *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2002. [Online]. Available: <https://dl.acm.org/doi/10.1145/564376.564429>
- [7] —, “A framework for selective query expansion,” in *Proceedings of the Thirteenth ACM International Conference on Information and Knowledge Management*, 2004. [Online]. Available: <https://dl.acm.org/doi/10.1145/1031171.1031220>
- [8] N. Rekabsaz, O. Lesota, M. Schedl, J. Brassey, and C. Eickhoff, “TripClick: The log files of a large health web search engine,” 2021. [Online]. Available: <https://tripdatabase.github.io/tripclick/>
- [9] —, “Establishing strong baselines for TripClick health retrieval,” in *European Conference on Information Retrieval*, 2022. [Online]. Available: <https://arxiv.org/abs/2201.00365>
- [10] SQLite Consortium, “SQLite FTS5 extension,” 2026. [Online]. Available: <https://www.sqlite.org/fts5.html>
- [11] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Pradeep, and R. Nogueira, “Pyserini: Reproducible IR research with sparse and dense representations,” 2026. [Online]. Available: <https://github.com/castorini/pyserini>
- [12] Hugging Face, “cross-encoder/ms-marco-MiniLM-L6-v2,” 2026. [Online]. Available: <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>